Memory tokens + mem→mem rollout training

for a Dreamer-4-style latent dynamics model

Merlin Richter

July 2026

Problem: the dynamics model is causal over a window of n_{ctx} frames — once an informative frame is evicted, its (possibly off-screen) state is gone. Goal: a bounded per-timestep **memory token** channel that carries hidden state indefinitely, trained so the model actually uses it.

Setup: token layout and attention

Two separate transformers (Dreamer-4 style):

- **Tokenizer** — video autoencoder, trained separately and then *frozen*; compresses each frame to latents $z_t \in \mathbb{R}^{n_z \times d_z}$. No denoising; not discussed further.
- **Dynamics model** f_θ — block-causal space/time transformer over the latents; the model of interest here. Predicts next-frame latents by denoising, trained with shortcut forcing.

Per frame t the dynamics transformer sees one token block:

$$[\ a_t \mid z_t\ (n_z) \mid \text{registers} \mid \underbrace{m_t\ (n_m)}_{\text{ours}} \mid \text{shortcut}(\tau_t, d_t)\ ]$$

- **Spatial layers:** full unmasked attention *within* a frame — memory $\leftrightarrow$ latents mix here.
- **Temporal layers** (every 3rd): causal in time, **per token slot** (RoPE on time). Each memory slot is its own causal channel: m_t's slot attends to $m_{<t}$, not to other slots at other times.
- **Memory tokens** m_t: final-layer activations, no ground-truth label. "Carrying" = repeatedly *read* old memory, *write* new memory — not threading a frozen activation forward.

Consequence: information can only cross time through the per-slot temporal channels; memory is the designated carrier for state that must outlive the window.

Base training: shortcut forcing (Dreamer-4, recap)

Per-frame signal level $\tau_t \in [0,1]$ and step size $d_t = 1/K$; clean latent z, Gaussian noise ε (nothing to do with the latents — pure noise):

$$\tilde{z} = (1-\tau)\,\varepsilon + \tau\, z, \qquad \hat{z} = f_\theta(\tilde{z}, \tau, d, a)$$

x-prediction (predict the clean $\hat{z}$, not velocity) — keeps long rollouts stable. Bootstrap distills two half-steps:

$$b' = \frac{f_\theta(\tilde{z}, \tau, \frac{d}{2}, a) - \tilde{z}}{1-\tau}, \quad z' = \tilde{z} + \tfrac{d}{2}\, b', \quad b'' = \frac{f_\theta(z', \tau+\frac{d}{2}, \frac{d}{2}, a) - z'}{1-(\tau+\frac{d}{2})}$$

$$\mathcal{L}_{\text{sf}} = w(\tau) \cdot \begin{cases} \|\hat{z} - z\|^2 & d = d_{\min} \\ (1-\tau)^2 \left\| \dfrac{\hat{z}-\tilde{z}}{1-\tau} - \text{sg}\left(\dfrac{b'+b''}{2}\right) \right\|^2 & \text{otherwise} \end{cases}$$

with ramp $w(\tau) = (1 - w_{\min})\tau + w_{\min}$. Inference denoises each frame in K=4 shortcut steps.

Memory tokens: what they are for

The window evicts: any state not re-observable from the last n_{ctx} frames is lost. The n_m memory tokens per timestep are a **bounded carrier** for that state — the target is an approximate **Markov state**, carried by *half a window* of memory:

$$p\big(z_{>t} \mid z_{\le t},\, a\big) \;=\; p\big(z_{>t} \mid m_{t-n_{\text{ctx}}/2\,:\,t},\, a_{>t}\big)$$

in words: for predicting future frames, the last $n_{\text{ctx}}/2$ memory tokens should be just as informative as the whole episode. (A *single* m_t is not meant to suffice — half a context window's worth of memory is.)

- m_t is an **activation** (final-layer hidden state), no ground-truth label — it can only be supervised *implicitly*, by making prediction depend on it.
- **Why plain windowed training gives no such signal:** every frame is predictable from the in-window latents, and the in-context memory tokens are always the learned blank init. Nothing forces the model to write useful memory — let alone to build m_t by reading $m_{<t}$ (**mem**→**mem**), which is exactly what carried inference requires.

Both signals come from one place: the rollout training on the next slide, whose *latent-hiding mode* makes memory the only path to the target.

mem→mem rollout training (ours)

Train on long clips with a sliding window instead of i.i.d. slices, so *real computed* memory sits in context:

1. Fixed window size n_{ctx} (currently 32); init window: near-clean GT latents, blank memory $\Rightarrow$ one forward pass writes real memory $m_{1..n_{\text{ctx}}}$.
2. Slide by $n_{\text{ctx}}/2$. **Old half:** carried (graph-attached) memory + latents drawn per batch element from one of two modes,

$$z^{\text{old}} \sim \tfrac{1}{2}\,\delta(\text{near-clean GT}) + \tfrac{1}{2}\,\delta(\underbrace{\text{pure noise, } \tau{=}0}_{\text{latent-hiding mode}})$$

   In latent-hiding mode the scene is gone from the latents — the new half is predictable *only* through the memory chain: the forcer that makes m encode the full state. **New half:** blank memory, noised latents.
3. Forward pass; **loss on the new half only**: the shortcut-forcing loss $\mathcal{L}_{\text{sf}}$ on the new frames. Old half is context, no loss.
4. Carry the new half's memory forward as the next old half; repeat until clip end.

Temporal attention is causal per slot, so new memory is *built by attending to old memory* — backprop relays new-m $\rightarrow$ old-m **construction** (TBPTT, detach at $2\,n_{\text{ctx}}$; no KV cache, full recompute). Verified: an evicted grounding latent gets nonzero grad through the relay, exactly 0 when detached.

Inference: carrying rollout

Autoregressive over a sliding window of n_{ctx} frames; context held near-clean at $\tau_{\text{ctx}} \approx 0.9$. Per new frame t:

1. **Denoise** from pure noise in K=4 shortcut steps ($d = 1/K$):

$$z^{(i+1)} = z^{(i)} + d \cdot \frac{f_\theta\big(z^{(i)}, \tau_i, d\big) - z^{(i)}}{1 - \tau_i}, \qquad \tau_i = i/K$$

   These steps only *read* the KV cache of committed past frames; m_t is taken from the last step.

2. **Commit** (5th pass): re-present frame t as it will be seen as context — latents at τ_{ctx} with the written m_t injected at the input — and store its K/V. (Fusable with the next frame's first denoise step.)

- **Memory relay:** each step reads the old memory tokens' cached K/V and writes a new m_t — state survives window eviction through repeated read→write, unboundedly.
- **RoPE at absolute rollout index** (never re-rotated; valid since RoPE is relative), so window eviction is a pure cache slice and rollouts exceed the trained window length.
- $n_m = 0$ recovers the vanilla Dreamer-4-style rollout.

Why the mem→mem signal matters (GridWorld probe)

6×6 GridWorld, square moving behind a full-frame curtain; recall = read out the square's position from a revealed prediction after k fully occluded rollout steps (chance = 1/36). Sliding window forced to 8 frames, k up to 64 — everything past $k \approx 8$ must live in memory. Position accuracy:

	$k = 4$ (in-window)	tail $k \geq 14$	$k = 64$
vanilla (no memory)	1.00	0.02	0.03
+ mem→mem rollout training	**0.98**	**0.99**	**0.98**

Vanilla is perfect while the last revealed frame is still in the window, then collapses to chance once it evicts; the mem→mem-trained model stays flat ≈ 1.0 to $k = 64$ — hidden state is carried indefinitely past the latent window. (64 env seeds per k; the result is stable across several independently trained mem→mem runs, all ≥ 0.97 on the tail.)